

Jonathan Steen - Assignment 10.2

May 18, 2025

0.0.1 Jonathan Steen

0.0.2 MSDS, Bellevue University

0.0.3 DSC 630: Predictive Analytics

0.0.4 Instructor: Frank Neugebauer

0.0.5 May 18th, 2025

0.0.6 10.2 Assignment: Recommender System

0.1 Set Imports, Load Data and Initial Exploration

```
[1]: # Imports
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from rapidfuzz import process

# Load the movie dataset
df = pd.read_csv('movies.csv')

# Display the first few rows
df.head()
```

```
[1]:      movieId      title \
0          1      Toy Story (1995)
1          2      Jumanji (1995)
2          3  Grumpier Old Men (1995)
3          4  Waiting to Exhale (1995)
4          5  Father of the Bride Part II (1995)

      genres
0  Adventure|Animation|Children|Comedy|Fantasy
1      Adventure|Children|Fantasy
2      Comedy|Romance
3  Comedy|Drama|Romance
4      Comedy
```

0.2 Step 1: Understand the Dataset and Basic Cleaning

There are three columns: movieId, title, and genres. I'll focus on genres to compare movies.

```
[2]: # Replace missing genres with empty string
df['genres'] = df['genres'].fillna('')
```

0.3 Step 2: Vectorize Genres with TF-IDF

```
[3]: # Convert genres to TF-IDF vectors
tfidf = TfidfVectorizer(token_pattern=r'[\^|]+' )
tfidf_matrix = tfidf.fit_transform(df['genres'])
```

Rationale: I turned genres into numbers so I can compare movies based on their genres. Since the genres are a pipe-separated list associated with each movie the `token_pattern=r'[\^|]+'` splits genres by the `|` symbol.

0.4 Step 3: Recommend Similar Movies

```
[4]: # Create a reverse mapping from movie title to index
title_to_index = pd.Series(df.index, index=df['title'])
```

```
[5]: # Find the closest matching title
def get_closest_title(input_title, choices, threshold=70):
    match = process.extractOne(input_title, choices, score_cutoff=threshold)
    return match[0] if match else None

# Define recommendation function with fuzzy title matching
def recommend_movies_fuzzy(input_title, num_recommendations=10,
    sample_size=10000):
    matched_title = get_closest_title(input_title, df['title'].tolist())
    if not matched_title:
        return f"No close match found for '{input_title}'."

    idx = title_to_index[matched_title]
    target_vector = tfidf_matrix[idx]

    # Randomly sample a subset of other movies
    sampled_indices = df.index[df.index != idx].to_series().sample(sample_size,
    random_state=42).tolist()
    sampled_matrix = tfidf_matrix[sampled_indices]

    # Compute cosine similarity between the target and sampled movies
    similarities = cosine_similarity(target_vector, sampled_matrix).flatten()
    top_indices = sorted(zip(sampled_indices, similarities), key=lambda x:
    x[1], reverse=True)[:num_recommendations]
    movie_indices = [i[0] for i in top_indices]
```

```
print(f"Showing results for: **{matched_title}**")
return df[['title', 'genres']].iloc[movie_indices]
```

```
[6]: # Enter Movie Title
      recommend_movies_fuzzy("Jumanji")
```

Showing results for: **Jumanji (1995)**

```
[6]:
```

		title	genres
56926	The Cave of the Golden Rose 5	(1996)	Adventure Children Fantasy
41321	Pete's Dragon	(2016)	Adventure Children Fantasy
44418	Perri	(1957)	Adventure Children Fantasy
75785	The Secret of the Iron Door	(1970)	Adventure Children Fantasy
83932	Slumberland	(2022)	Adventure Children Fantasy
26317	Le petit poucet	(2001)	Adventure Children Fantasy
71372	The Christmas Chronicles: Part Two	(2020)	Adventure Children Fantasy
79179	A Boy Called Christmas	(2021)	Adventure Children Fantasy
67606	Foxter and Max	(2019)	Adventure Children Fantasy
70387	The Legend of The Five	(2020)	Adventure Children Fantasy